

Software Design Document

1. Introduzione

Il presente documento descrive l'architettura software del progetto Music Playlist Manager. Il sistema consiste in un'applicazione desktop Java per la creazione e la gestione di playlist musicali. L'applicazione permette all'utente di gestire una libreria di tracce, organizzarle in playlist e simulare la riproduzione dei brani attraverso comandi come play, pause e skip.

Le funzionalità principali previste sono:

- gestione delle tracce musicali;
- creazione, modifica, eliminazione e visualizzazione delle tracce;
- gestione delle playlist;
- aggiunta e rimozione di tracce da playlist;
- playback simulato di tracce e playlist;
- supporto alle modalità sequenziale, shuffle e loop;
- aggiornamento della UI durante il playback;
- possibile estensione con undo, statistiche, tag, filtri e smart playlist.

Il documento adotta il modello "4+1" di Kruchten, che descrive l'architettura attraverso cinque prospettive:

1. **Vista Logica**, per descrivere le astrazioni principali del sistema;
2. **Vista di Processo**, per descrivere il comportamento a runtime;
3. **Vista di Sviluppo**, per descrivere l'organizzazione del codice;
4. **Vista Fisica**, per descrivere il deployment;
5. **Scenari "+1"**, per validare l'architettura rispetto ai casi d'uso principali.

L'obiettivo del documento è fornire in modo sintetico e chiaro le principali decisioni di design, i componenti ad alto livello, le interfacce tra i moduli, i flussi di dati, la strategia di gestione degli errori e i pattern architetturali e di design adottati.

2. Vista Logica

La Vista Logica descrive la struttura funzionale del sistema e le principali astrazioni del dominio applicativo. In questa vista vengono identificati i moduli logici, le classi principali, le loro responsabilità e le relazioni fondamentali tra gli oggetti.

Nel nostro caso, il dominio ruota attorno a tre concetti principali:

- la **libreria musicale**, composta da tracce;
- le **playlist**, che organizzano le tracce;

- il **player simulato**, che gestisce lo stato della riproduzione.

2.1 Componenti logici principali

A livello logico, il sistema può essere suddiviso nei seguenti componenti principali.

Componente	Responsabilità
Modulo FXML / JavaFX	Gestisce la presentazione dell'interfaccia grafica tramite viste definite in file FXML. Visualizza i dati e permette all'utente di interagire con il sistema.
Modulo Controller	Coordina le richieste provenienti dalla UI e delega le operazioni ai servizi applicativi. Mantiene separata la logica di presentazione dalla logica di dominio.
Modulo Modello (Domain Model)	Contiene le entità principali del sistema: <code>Track</code> , <code>Playlist</code> , <code>MusicLibrary</code> e <code>TrackTag</code> . Rappresenta i dati e le relazioni del dominio musicale.
Modulo Servizi (Business Logic)	Gestisce la logica aziendale pesante. <code>TrackService</code> (validazione tracce), <code>PlaylistService</code> (creazione, filtri e sincronizzazione Smart Playlist), <code>AudioPlayerService</code> (riproduzione simulata e aggiornamento timer), <code>MusicLibraryService</code> e relativi package contenuti internamente come <code>command</code> , <code>playback</code> , <code>playlistnavigation</code> , <code>persistenc</code> e <code>statistics</code> .
Modulo Command (Undo/Redo)	Gestisce l'incapsulamento delle operazioni distruttive in oggetti comando (<code>DeleteTrackCommand</code> , <code>AssignTrackCommand</code>) per supportare la funzionalità di "Annulla" tramite il <code>CommandManager</code> .

Modulo Playback & Navigazione	Definisce le modalità di riproduzione tramite <code>PlaybackMode</code> e le strategie di navigazione playlist tramite <code>PlaylistNavigationStrategy</code> .
Modulo Statistics	Fornisce la logica per la schermata delle statistiche delle canzoni e playlist <code>PlaybackStatisticsService</code> .
Modulo Repository	Fornisce l'astrazione per il salvataggio e caricamento dei dati tramite <code>TrackRepository</code> , <code>PlaylistRepository</code> e <code>MusicLibraryRepository</code> .
Modulo Persistence	Gestisce le operazioni di caricamento e salvataggio asincrono dei dati tramite <code>PersistenceService</code> .
Modulo exception	Contiene le eccezioni applicative e supporta la gestione degli errori generati dai servizi.
Modulo util	Contenente la classe <code>AlertUtili</code> per la gestione degli alert

Il sistema segue una logica a responsabilità separate. La GUI non deve contenere direttamente la logica di business; essa comunica con i controller, che a loro volta invocano i servizi applicativi.

Schema logico di alto livello:

None

`Utente`



`Modulo GUI`



`Controller`

↓

Service Layer

↓

Domain Model / Repository

2.2 Classi principali del dominio

Le principali classi del dominio sono le seguenti.

Track

Rappresenta una singola traccia musicale.

Attributi principali:

None

-id

-title

-author

-length

-genre

-year

-tags

-playCount

La traccia è l'entità base dell'applicazione. Contiene le informazioni descrittive del brano come titolo, autore, durata, genere e anno di pubblicazione.. In estensioni successive può essere arricchita con tag visivi come favourite, explicit e new release.

Playlist

Rappresenta una raccolta ordinata di tracce.

Attributi principali:

None

- `id`
- `name`
- `tracks`
- `playCount`
- `generataAutomaticamente`
- `tipoFiltro`
- `filtroAutomatico`

Operazioni principali:

None

- + `addTrack(track)`
- + `removeTrack(track)`
- + `getTracks()`
- + `incrementPlayCount()`

Una playlist contiene riferimenti a tracce presenti nella libreria. Rimuovere una traccia da una playlist non significa eliminarla dalla libreria generale.

La classe supporta anche playlist automatiche generate tramite filtri (ad esempio genere o anno).

MusicLibrary

Rappresenta il contenitore principale dei dati dell'applicazione.

Attributi principali:

None

- `tracks`
- `playlists`

Operazioni principali:

None

+ `getAllTracks()`

+ `getPlaylists()`

Questa classe separa la gestione dello stato del player dalla GUI. L'interfaccia mostra lo stato, ma non lo gestisce direttamente.

PlaybackMode

Rappresenta la strategia con cui viene scelta la prossima traccia.

Implementazioni previste:

None

`SequentialPlaybackMode`

`ShufflePlaybackMode`

`LoopPlaybackMode`

Questa astrazione consente di gestire modalità diverse di playback senza concentrare tutta la logica dentro un unico metodo pieno di condizioni.

TrackTag

Rappresenta eventuali tag visivi associabili alle tracce.

Valori previsti:

None

`FAVOURITE`

`EXPLICIT`

`NEW_RELEASE`

2.3 Relazioni tra le classi

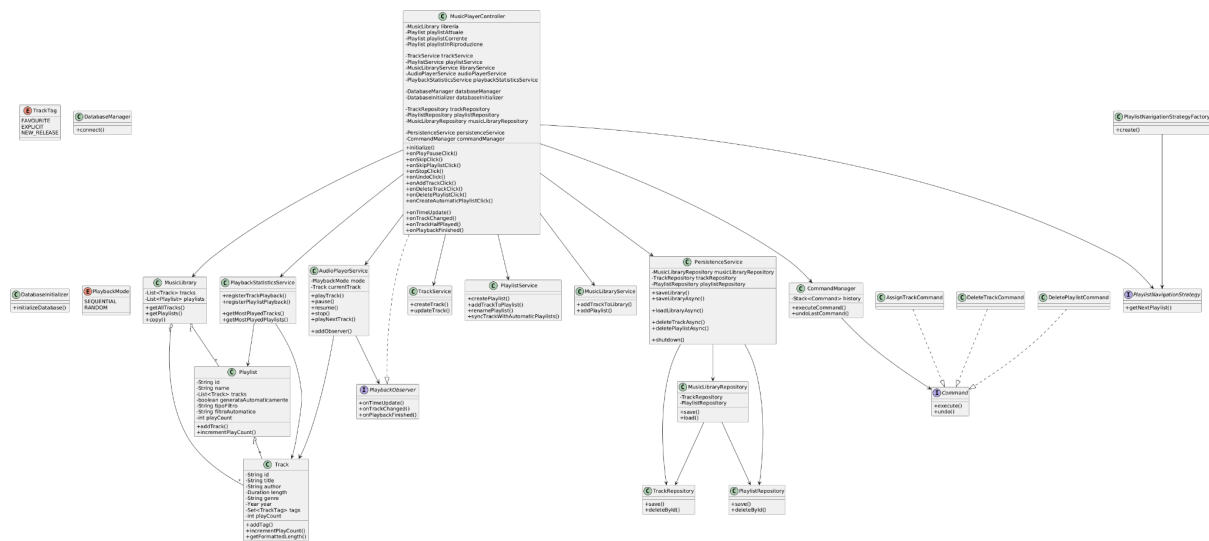
Le relazioni principali sono:

1. **Una `MusicLibrary` contiene zero o più `Track` e zero o più `Playlist`.** La libreria rappresenta il contenitore principale dei dati dell'applicazione e mantiene tutte le tracce e playlist disponibili.
2. **Una `Playlist` contiene zero o più `Track`.** Una playlist mantiene una collezione ordinata di riferimenti alle tracce presenti nella libreria.
3. **Una `Track` può appartenere a più `Playlist`.** La relazione è molti-a-molti: la stessa traccia può essere presente in playlist differenti senza essere duplicata nella libreria.
4. **Una `Track` può avere zero o più `TrackTag`.** I tag permettono di classificare le tracce, ad esempio come preferite, esplicite o nuove uscite.
5. **`AudioPlayerService` mantiene il riferimento alla traccia attualmente in riproduzione.** Il servizio audio gestisce lo stato del playback e comunica gli aggiornamenti alla GUI tramite il pattern Observer.
6. **`AudioPlayerService` utilizza un `PlaybackMode` per determinare la modalità di riproduzione.** La modalità influenza il comportamento del player durante la scelta della prossima traccia.
7. **`PlaybackMode` è un'enumerazione che rappresenta le modalità disponibili.**
8. **`PlaylistNavigationStrategy` definisce la strategia di selezione della prossima playlist.**
9. **`MusicPlayerController` comunica con i servizi applicativi (`AudioPlayerService`, `PlaylistService`, `TrackService`, ecc.).**
Il controller coordina le operazioni richieste dall'utente e aggiorna la GUI.
10. **`Repository` gestiscono la persistenza delle entità del modello.**
`TrackRepository`, `PlaylistRepository` e `MusicLibraryRepository` permettono il salvataggio e il caricamento dei dati.

Questa struttura favorisce la separazione di concetti come:

- dato musicale;
- organizzazione dei brani;
- stato del player;
- strategia di riproduzione.

2.4 Diagramma delle classi



2.5 Pattern rilevanti nella vista logica

Nella vista logica emergono soprattutto due pattern:

- Il primo è lo **Strategy Pattern**, applicato alla gestione delle modalità di playback. Il comportamento di scelta della prossima traccia cambia a seconda della modalità attiva: sequenziale, shuffle o loop. Incapsulare questi comportamenti in classi separate evita di concentrare tutto nel **PlaybackService**.
- Il secondo è il **Repository Pattern**. Il dominio e i servizi non devono dipendere dal modo concreto in cui tracce e playlist vengono salvate. Nella prima versione i dati possono essere mantenuti in memoria, ma in futuro sarà possibile introdurre persistenza locale senza modificare la logica applicativa.
- **Command Pattern (Gestione dell'Undo)**: Le azioni dell'utente, come l'eliminazione di tracce/playlist o l'assegnazione a playlist, sono incapsulate in classi comando (es. **DeleteTrackCommand**). Ogni comando sa come "eseguirsi" sul database in memoria e come "annullarsi", ripristinando gli indici esatti in cui le tracce si trovavano. Il **CommandManager** gestisce l'esecuzione e lo storico.

3. Vista di Processo

La Vista di Processo descrive il comportamento del sistema a runtime. In particolare, mostra come i componenti collaborano durante l'esecuzione, come fluiscono i dati tra i moduli, come viene gestito il playback simulato e come vengono trattati gli errori.

Essendo una desktop application, non prevede distribuzione del carico su più nodi. Tuttavia, è comunque importante descrivere il flusso degli eventi e la sincronizzazione tra stato interno e interfaccia grafica.

3.1 Comportamento runtime generale

Il comportamento generale del sistema è **event-driven**.

L'utente interagisce con la GUI, ad esempio cliccando su tasti come "Create Track", "Add to Playlist", "Play" o "Skip". La View raccoglie l'evento e lo inoltra al Controller. Il Controller invoca il Service appropriato. Il Service applica la logica applicativa, valida i dati, aggiorna il modello di dominio e usa il Repository se è necessario salvare o recuperare informazioni.

Flusso generale:

None

Utente → View → Controller → Service → Domain / Repository

Il risultato dell'operazione segue il percorso inverso:

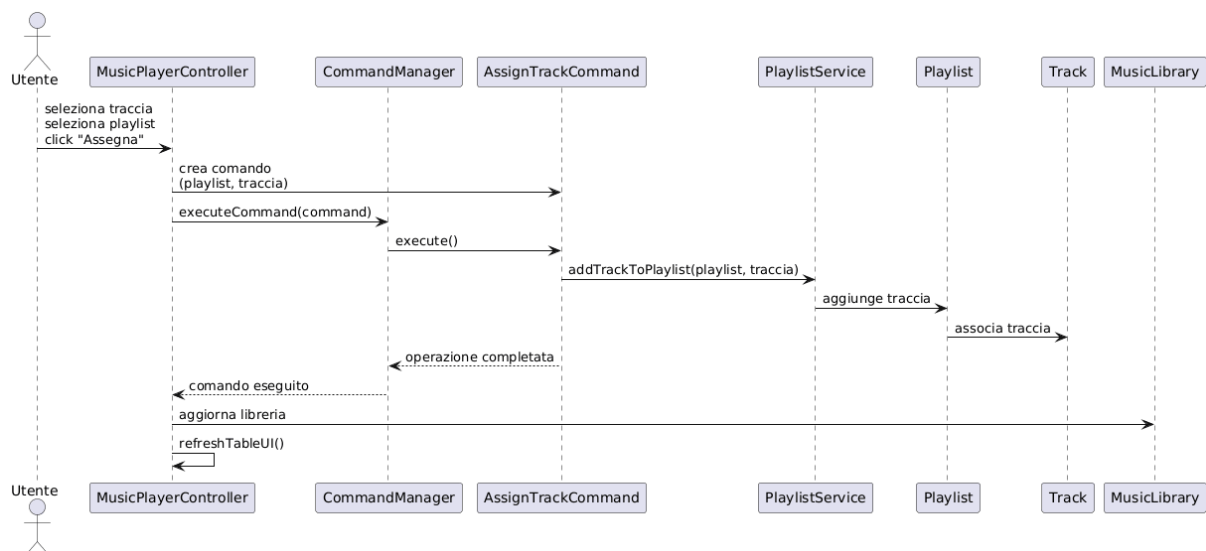
None

Service → Controller → View → Utente

Questa struttura rispetta la separazione delle responsabilità e permette di testare la logica applicativa senza dipendere dalla GUI.

3.2 Flusso dei dati e interazioni

Un esempio rappresentativo è l'aggiunta di una traccia ad una playlist.



1. L'utente seleziona una traccia e una playlist dall'interfaccia grafica e richiede l'assegnazione della traccia alla playlist.
2. La View intercetta l'azione dell'utente e la comunica al **MusicPlayerController**.

3. Il **MusicPlayerController** recupera la traccia e la playlist selezionate e crea un nuovo comando di assegnazione (**AssignTrackCommand**) contenente gli elementi necessari all'operazione.
4. Il controller delega l'esecuzione del comando al **CommandManager**, che invoca il metodo `execute()` del comando.
5. L'**AssignTrackCommand** richiede al **PlaylistService** di eseguire l'operazione di aggiunta della traccia alla playlist.
6. Il **PlaylistService** applica la logica applicativa necessaria e invoca il metodo `addTrack()` sulla classe **Playlist**.
7. La classe **Playlist** aggiorna la propria collezione interna di tracce associando il nuovo brano e comunica il completamento dell'operazione.
8. Il controllo ritorna al **PlaylistService**, che comunica al comando l'avvenuta esecuzione.
9. Il **CommandManager** conclude l'esecuzione del comando e restituisce il controllo al **MusicPlayerController**.
10. Infine il **MusicPlayerController** aggiorna la View mostrando il nuovo contenuto della playlist.

Flusso sintetico:

None

Utente

↓

MusicPlayerController

↓

AssignTrackCommand

↓

CommandManager

↓

PlaylistService

↓

Playlist

↓

CommandManager

↓

MusicPlayerController

↓

Aggiornamento GUI

Lo stesso principio vale per playlist e playback.

Le informazioni non devono passare direttamente dalla GUI al repository, perché questo favorirebbe un alto accoppiamento tra interfaccia e gestione dati.

3.3 Gestione del playback simulato

Il playback viene gestito dal modulo `playback` e dal servizio `AudioPlayerService`.

Il sistema non effettua una riproduzione audio reale, ma simula il comportamento di un lettore musicale mantenendo lo stato corrente del player.

Il componente principale è `AudioPlayerService`, che si occupa di:

- mantenere la traccia attualmente riprodotta;
- gestire lo stato del player;
- controllare la playlist corrente;
- applicare la modalità di riproduzione selezionata;
- notificare i componenti interessati quando cambia lo stato del playback.

Il comportamento della riproduzione viene definito tramite il pattern **Strategy Pattern**.

Quando l'utente avvia il playback di una playlist, il `PlaybackService` seleziona la traccia da riprodurre attraverso la strategia di playback attiva.

Il flusso logico è:

None

Controller → AudioPlayerService → PlaybackStrategy → Playlist / Track

La modalità di playback determina il comportamento:

Modalità	Comportamento
Sequential	Le tracce vengono riprodotte nell'ordine della playlist.
Shuffle	La prossima traccia viene scelta casualmente.
Repeat One	Riproduce nuovamente la traccia corrente senza avanzare.

Il sistema mantiene quindi lo stato della riproduzione e simula le operazioni principali di un player musicale (**PLAY**, **PAUSE**, **STOP**, cambio traccia), permettendo alla UI di riflettere sempre la situazione corrente.

3.4 Aggiornamento della UI

Il sistema utilizza un meccanismo di notifica basato sull'**Observer Pattern** per sincronizzare automaticamente l'interfaccia grafica con lo stato del player.

Quando **AudioPlayerService** modifica il proprio stato interno, ad esempio:

- cambio della traccia corrente;
- avvio della riproduzione;
- pausa;
- arresto del playback;

viene generato un evento verso gli osservatori registrati tramite **PlaybackObserver**.

Il flusso previsto è:

None

AudioPlayerService

↓

PlaybackObserver

↓

MusicPlayerController

↓

Interfaccia JavaFX (FXML)

↓

Aggiornamento UI

Questa architettura mantiene separata la logica di riproduzione dalla presentazione grafica:

- `AudioPlayerService` gestisce il comportamento del player;
- `PlaybackObserver` comunica le variazioni;
- il controller aggiorna gli elementi della GUI.

L'utilizzo dell'Observer Pattern permette inoltre di aggiungere nuovi componenti interessati agli eventi del player senza modificare il codice responsabile della riproduzione, migliorando modularità ed estendibilità del sistema.

3.5 Gestione degli errori a runtime

Durante l'esecuzione possono verificarsi errori o input non validi. Alcuni esempi sono:

- titolo della traccia vuoto;
- autore vuoto;
- durata negativa o nulla;
- anno non valido;
- nome playlist vuoto;
- tentativo di riprodurre una playlist vuota;
- tentativo di fare skip senza una traccia successiva;
- tentativo di rimuovere da una playlist una traccia non presente.

La gestione degli errori segue questo flusso:

None

`Service` → eccezione applicativa → `Controller` → messaggio utente nella View

Le eccezioni applicative previste sono:

None

`InvalidTrackException`

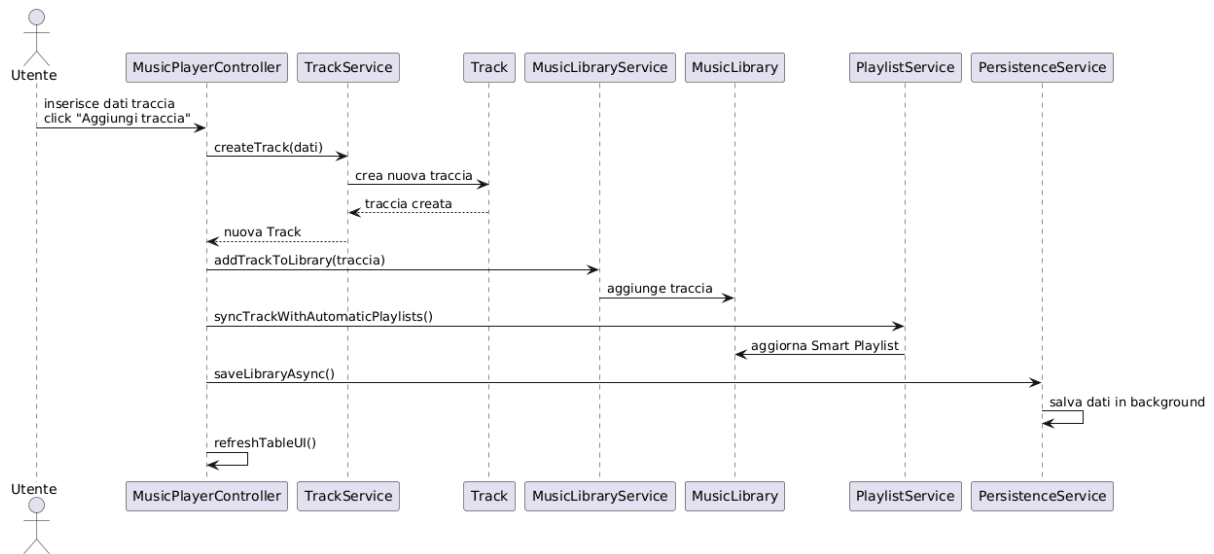
`InvalidPlaylistException`

PlaybackException

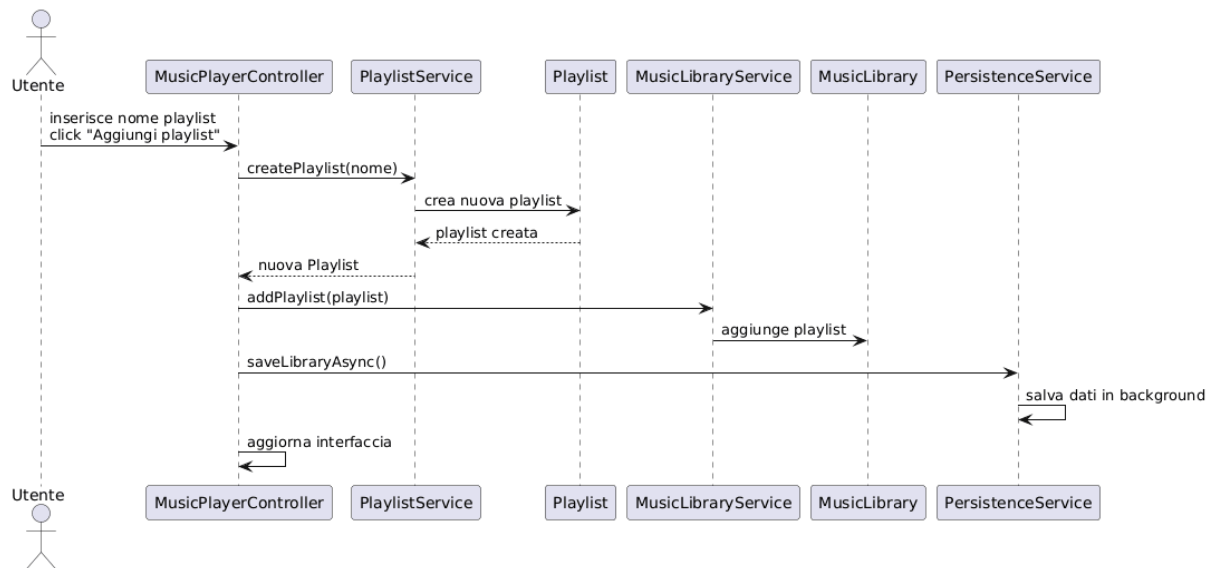
Il Service è responsabile della validazione e della generazione dell'errore.
Il Controller intercetta l'errore e lo converte in un messaggio comprensibile per l'utente.
In questo modo, dettagli tecnici interni non vengono mostrati direttamente nella GUI.

3.6 Diagrammi di sequenza

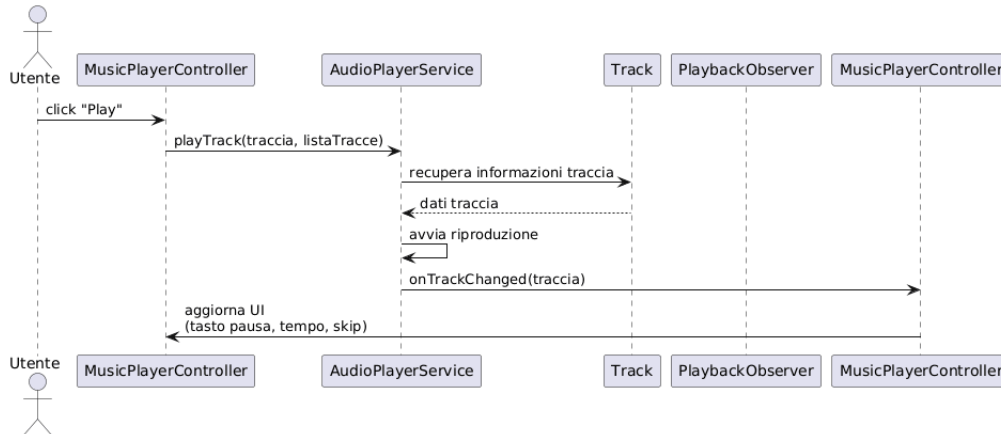
Creazione di una traccia



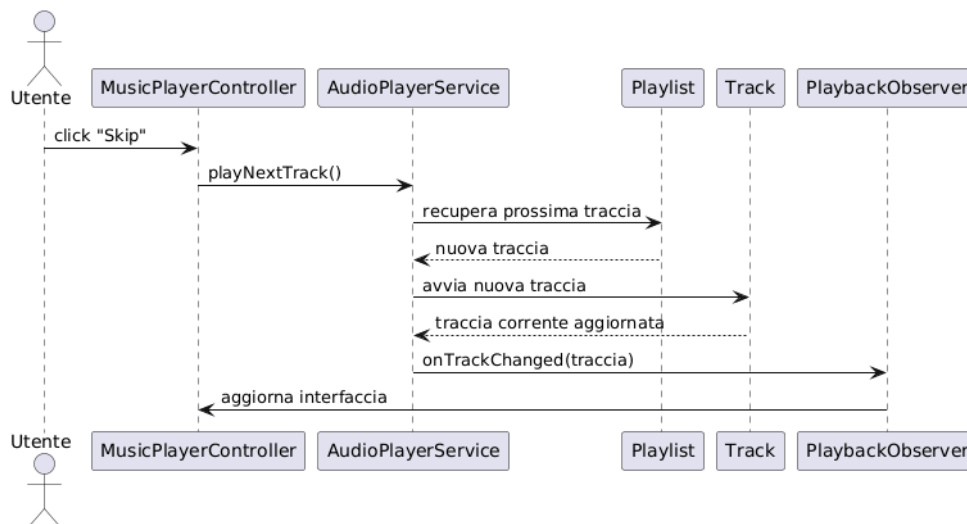
Creazione di una playlist



Avvio del playback



Skip della traccia corrente



4. Vista di Sviluppo

La Vista di Sviluppo descrive come il software è stato organizzato nel codice sorgente, rendendo esplicita la struttura dei package, dei moduli fisici, delle librerie e dei test.

4.1 Organizzazione dei package

La struttura proposta dei package Java è la seguente:

Con questa organizzazione abbiamo cercato di mirare ad un'architettura a livelli e permette di separare la presentazione, la logica applicativa, il dominio e l'accesso ai dati.

4.2 Moduli fisici del codice

I principali moduli fisici sono:

Package	Responsabilità
<code>controller</code>	Gestione degli eventi provenienti dall'interfaccia grafica e coordinamento tra UI e servizi applicativi. Contiene controller JavaFX come <code>MusicPlayerController</code> e <code>StatisticsController..</code>
<code>model</code>	Contiene le entità principali del dominio dell'applicazione, come <code>Track</code> , <code>Playlist</code> , <code>MusicLibrary</code> e <code>TrackTag</code> . Rappresenta lo stato dei dati gestiti dal sistema.
<code>service</code>	Contiene la logica applicativa dell'applicazione. Include servizi come <code>TrackService</code> , <code>PlaylistService</code> , <code>MusicLibraryService</code> e <code>AudioPlayerService</code> .
<code>command</code>	Implementa il Command Pattern per incapsulare operazioni eseguibili, come aggiunta, modifica o eliminazione di elementi tramite comandi separati.
<code>playback</code>	Gestisce il comportamento del player musicale. Contiene strategie di riproduzione (<code>SequentialPlaybackStrategy</code> , <code>ShufflePlaybackStrategy</code> , <code>RepeatOnePlaybackStrategy</code>), osservatori e modalità di playback.
<code>playlistnavigation</code>	Gestisce la navigazione tra le playlist tramite strategie dedicate.

<code>repository</code>	Contiene i componenti responsabili dell'accesso ai dati persistenti, separando la logica di memorizzazione dal resto dell'applicazione.
<code>persistence</code>	Gestisce il salvataggio e il caricamento dei dati tramite servizi di persistenza e gestione del database.
<code>statistics</code>	Contiene la logica relativa alla raccolta e gestione delle statistiche di riproduzione.
<code>exception</code>	Contiene eccezioni personalizzate utilizzate per gestire errori controllati dell'applicazione, come errori di validazione.
<code>util</code>	Contiene classi di supporto e utilità generiche.
<code>test</code>	Contiene i test automatici delle componenti principali del sistema.

4.3 Dipendenze tra package

Le dipendenze principali seguono una struttura a livelli:

```

None
FXML / JavaFX View

↓

controller

↓

service

↓

```

```
model
↓
repository
↓
persistence
```

Il flusso applicativo è il seguente:

- La parte grafica JavaFX definita tramite file FXML viene collegata ai relativi controller.
- I controller ricevono gli eventi dell'utente e richiamano i servizi necessari.
- I service applicano le regole di business e coordinano le operazioni sulle entità del modello.
- Il model rappresenta i dati principali dell'applicazione.
- I repository astraggono la gestione dei dati persistenti.
- Il livello persistence si occupa dell'effettivo salvataggio e caricamento.

Sono presenti inoltre moduli trasversali:

- Il package `playback` utilizza l'Observer Pattern per notificare ai componenti interessati i cambiamenti dello stato del player.
- Il package `service.command` utilizza il Command Pattern per separare le richieste operative dalla loro esecuzione.
- Il package `playback` e `playlistnavigation` utilizzano lo Strategy Pattern per permettere di cambiare dinamicamente il comportamento di riproduzione e navigazione.

Le principali regole architetturali adottate sono:

- I controller non contengono logica di business complessa.
- I service sono responsabili delle regole applicative.
- Il model non dipende dalla GUI.
- I repository non conoscono i controller o i componenti grafici.
- La persistenza è separata dalla logica applicativa.
- Le eccezioni personalizzate vengono utilizzate per comunicare errori previsti dal sistema.

Abbiamo adottato questa struttura in quanto riduce l'accoppiamento e supporta il principio di separazione delle responsabilità.

4.4 Librerie e strumenti di build

Gli strumenti previsti sono:

Strumento	Uso
Java	Linguaggio di sviluppo dell'applicazione.
GitHub	Repository del progetto e versionamento.
JUnit	Test unitari.
Trello	Tracciamento attività, sprint backlog e task board.
Maven	Gestione build e dipendenze, da confermare dal team.

4.5 Organizzazione dei test

I test saranno collocati in una struttura separata dal codice principale.

Esempio:

None

`src/main/java`

`src/test/java`

I test dovrebbero concentrarsi principalmente sulla business logic:

- validazione delle tracce;
- creazione delle playlist;
- aggiunta e rimozione di tracce;
- comportamento delle modalità di playback;
- gestione degli errori applicativi.

Esempi di classi di test:

None

TrackServiceTest

PlaylistServiceTest

MusicLibraryServiceTest

CommandManagerTest

La GUI può essere verificata con test manuali o, se il team lo ritiene opportuno, con test specifici in una fase successiva. Per la valutazione iniziale è importante che almeno le classi di business logic siano testabili in modo automatico.

5. Vista Fisica

La Vista Fisica descrive come il software viene distribuito sull'infrastruttura hardware e nel nostro caso, essendo un'applicazione desktop stand-alone, il deployment non sarà molto complesso.

5.1 Ambiente di esecuzione

L'applicazione viene eseguita localmente sul computer dell'utente.

Ambiente previsto:

None

PC / Laptop dell'utente

Sistema operativo desktop

Java Virtual Machine

Music Playlist Manager

5.2 Gestione iniziale dei dati

Il sistema utilizza un livello di persistenza basato su database per il salvataggio dei dati dell'applicazione.

I dati principali, come tracce, playlist e libreria musicale, vengono salvati automaticamente attraverso il livello **persistence** e rimangono disponibili anche dopo la chiusura e il riavvio dell'applicazione.

I repository (**TrackRepository**, **PlaylistRepository**, **MusicLibraryRepository**) forniscono un'astrazione per l'accesso ai dati, evitando che la logica applicativa dipenda direttamente dal sistema di memorizzazione.

Questa scelta permette di mantenere separata la gestione dei dati dalla logica del programma: i servizi applicativi operano sui repository senza conoscere i dettagli del database utilizzato.

Il sistema quindi supporta la conservazione dello stato tra sessioni diverse, permettendo all'utente di ritrovare tracce, playlist e configurazioni precedentemente salvate anche dopo la chiusura dell'applicazione.

5.3 Possibili estensioni future

Possibili evoluzioni del deployment sono:

- esportazione/importazione di playlist;
- sincronizzazione cloud;
- gestione multiutente.

6. Scenari “+1”

Gli scenari “+1” servono a validare le altre viste architetture. Mostrano come la Vista Logica, la Vista di Processo, la Vista di Sviluppo e la Vista Fisica cooperano per realizzare casi d'uso concreti.

6.1 Scenario: creazione di una traccia

L'utente inserisce titolo, autore, durata, genere e anno di una nuova traccia.

Flusso:

None

Utente → MusicPlayerController → TrackService → Track →
MusicLibraryService → TrackRepository

Componenti coinvolti:

- MusicPlayerController;
- TrackService;
- Track;
- MusicLibrary;
- MusicLibraryService;
- TrackRepository.

Validazione:

- titolo non vuoto;
- autore non vuoto;
- durata positiva;
- anno valido.

Pattern coinvolti:

- MVC;
- Service Layer;
- Repository Pattern.

6.2 Scenario: creazione di una playlist

L'utente inserisce il nome di una nuova playlist.

Flusso:

None

Utente → MusicPlayerController → Playlist → PlaylistService →
MusicLibraryService → TrackRepository

Componenti coinvolti:

- MusicPlayerController;
- PlaylistService;
- Playlist;
- MusicLibrary;
- MusicLibraryService;
- PlaylistRepository.

Validazione:

- nome playlist non vuoto;
- playlist creata inizialmente senza tracce.

Pattern coinvolti:

- MVC;
- Service Layer;
- Repository Pattern.

6.3 Scenario: aggiunta traccia a playlist

L'utente seleziona una traccia dalla libreria e una playlist di destinazione.

Flusso:

None

Utente → MusicPlayerController → AssignTrackCommand → PlaylistService → Playlist

Il **PlaylistService** recupera traccia e playlist, poi aggiorna la playlist.

La traccia non viene rimossa dalla libreria generale: viene soltanto associata alla playlist.

Componenti coinvolti:

- MusicPlayerController;
- CommandManager;
- AssignTrackCommand;
- PlaylistService;
- Playlist;
- Track.

Pattern coinvolti:

- MVC;
- Command Pattern;
- Service Layer;
- Repository Pattern.

6.4 Scenario: playback simulato

L'utente seleziona una playlist e clicca su Play.

Flusso:

None

Utente → MusicPlayerController → AudioPlayerService → PlaybackMode → PlaybackObserver → Aggiornamento GUI

Componenti coinvolti:

- MusicPlayerController;
- AudioPlayerService;
- PlaybackMode;
- PlaybackObserver;
- Track;
- Playlist.

Pattern coinvolti:

- MVC;
- Strategy Pattern;
- Observer Pattern.

6.5 Scenario: gestione input non valido

L'utente prova a creare una traccia con un anno non valido o una durata negativa.

Flusso:

None

Utente → MusicPlayerController → AudioPlayerService →
PlaylistNavigationStrategy → nuova Track → Aggiornamento GUI

Il controller riceve l'azione di skip e la inoltra al servizio audio.

AudioPlayerService utilizza la strategia di navigazione corrente per individuare la traccia successiva.

Il player aggiorna la traccia corrente e comunica il cambiamento alla GUI tramite gli observer registrati.

Pattern coinvolti:

- MVC;
- Strategy Pattern;
- Observer Pattern.

6.6 Validazione delle viste attraverso gli scenari

Scenario	Vista logica	Vista processo	Vista sviluppo	Vista fisica
Creazione traccia	Track e gestione libreria	richiesta utente → Controller → TrackService → Repository → salvataggio dati	package model, service, repository , persistenc e	JVM locale + database persistente
Creazione playlist	Playlist e relazione con Track	validazione → PlaylistService → aggiornamento libreria → persistenza	package model, service, repository	JVM locale + database persistente
Aggiunta traccia	relazione Track-Play list	Controller → PlaylistService → modifica stato playlist → salvataggio	package controller , service, model	JVM locale + database persistente
Playback	AudioPlaye rService, PlaybackMo de, PlaybackSt rategy	stato runtime player → selezione strategia → notifica osservatori	package playback, service	JVM locale
Cambio stato player	stato playback e traccia corrente	AudioPlayerServic e → PlaybackObserver → Controller → aggiornamento UI	package playback, controller	JVM locale

Navigazione playlist	strategia di navigazione playlist	richiesta cambio traccia → PlaylistNavigationStrategy	package playlistnavigation	JVM locale
Input non valido	gestione errori applicativi	validazione Service → lancio eccezione → gestione Controller	package exception, service	nessun impatto fisico
Caricamento dati	stato persistente libreria	avvio applicazione → PersistenceService → Repository → Database	package persistence, repository	Database persistente
Statistiche playback	dati statistici riproduzione	evento playback → aggiornamento statistiche → visualizzazione	package statistics, service, controller	JVM locale + database

7. Decisioni architetturali principali

Decisione	Motivazione
Applicazione desktop Java	Richiesta dalla traccia del progetto.
Architettura a livelli	Separa GUI, logica applicativa, dominio e dati.

MVC	Disaccoppia interfaccia grafica e logica applicativa.
Repository Pattern	Astrae la gestione dei dati tramite interfacce indipendenti dal sistema di persistenza.
Persistence Layer separato	Gestisce l'accesso al database SQLite e isola la logica JDBC dal resto dell'applicazione.
Strategy Pattern per playback e playlistnavigation	Permette di gestire modalità diverse senza modificare il player.
Observer Pattern per UI update	Aggiorna la UI quando cambia lo stato del playback.
Factory Pattern per playback e playlistnavigation	Crea e restituisce l'implementazione corretta di Strategy (Playback e PlaylistNavigation) in base al PlaybackMode selezionato
Command Pattern per undo	Supporta operazioni annullabili.
JUnit per test	Permette di verificare automaticamente la business logic.
Trello e GitHub	Supportano tracciamento Scrum e versionamento.

8. Design Pattern utilizzati

Architettura a livelli

Il sistema viene diviso in livelli:

None

View → Controller → Service → Domain / Repository

Questa scelta mira a migliorare la separazione delle responsabilità, manutenibilità e testabilità.

Model-View-Controller

MVC viene usato per la gestione della GUI.

- **Model:** Track, Playlist, MusicLibrary;
- **View:** schermate dell'applicazione;
- **Controller:** classi che gestiscono gli input dell'utente.

MVC evita che la logica di business venga inserita direttamente nella GUI.

Repository Pattern

Il Repository Pattern isola il Service Layer dai dettagli di memorizzazione.

None

TrackService → TrackRepository → Database-backed Repository
tramite JDBC

Il Repository Pattern astrae l'accesso ai dati, permettendo di cambiare l'implementazione del persistenza senza modificare la logica di business.

Strategy Pattern

Lo Strategy Pattern viene utilizzato per incapsulare gli algoritmi di playback e playlistnavigation.

None

Playback

└─ SequentialPlaybackStrategy

└─ ShufflePlaybackStrategy

└─ RepeatOnePlaybackStrategy

Playlistnavigation

```
|— RepeatOnePlaylistNavigationStrategy
|— ShufflePlaylistNavigationStrategy
|— SequentialPlaylistNavigationStrategy
```

Ogni strategia definisce il comportamento per la selezione della prossima traccia.

Factory Pattern

Il Factory Pattern viene utilizzato per creare e restituire la strategia di playback e playlistnavigation corretta in base a quella selezionata.

Componenti software interessati:

- `PlaybackStrategyFactory`
- `PlaylistNavigationStrategyFactory`
- `PlaybackMode` (tipo enumerativo impiegato per la scelta della strategia)

Observer Pattern

Observer viene implementato tramite l'interfaccia `PlaybackObserver` ed è usato per aggiornare la UI quando cambia lo stato del playback.

```
None
PlaybackService → PlaybackObserver (Controller JavaFX) →
aggiornamento View
```

Il vantaggio è evitare polling continuo e ridurre l'accoppiamento tra player e GUI.

Command Pattern

Il Command Pattern viene utilizzato per incapsulare operazioni sul dominio (playlist e tracce), permettendo la loro esecuzione tramite `CommandManager`.

```
None
command
|— AssignTrackCommand
```

```
|— DeletePlaylistCommand
|— DeleteTrackCommand

CommandManager (gestore separato)
```

Ogni comando può esporre:

```
None
execute()

undo()
```

Questo permette di annullare rimozioni di tracce da playlist.

9. Strategia di gestione degli errori

La strategia di gestione degli errori si basa su tre livelli:

1. validazione nel Service Layer;
2. eccezioni applicative specifiche;
3. messaggi utente gestiti dalla View.

Errori previsti

Errore	Gestione
Titolo vuoto	<code>InvalidTrackException</code>
Autore vuoto	<code>InvalidTrackException</code>
Durata non valida	<code>InvalidTrackException</code>

Anno non valido	<code>InvalidTrackException</code>
Nome playlist vuoto	<code>InvalidPlaylistException</code>
Playlist vuota in playback	<code>PlaybackException</code>
Skip non possibile	<code>PlaybackException</code>
Rimozione traccia assente	errore controllato o operazione ignorata in modo sicuro

Flusso di gestione errore

None

`Service`

↓ `lancia eccezione`

`Controller`

↓ `intercetta eccezione`

`View`

↓ `mostra messaggio`

`Utente`

Esempio:

None

`InvalidTrackException`

→ `"La durata della traccia deve essere maggiore di zero."`

Abbiamo adottato questa scelta per evitare di mostrare dettagli tecnici all'utente e mantenere separata la logica di validazione dalla logica di presentazione.

10. "Definition of Done"

Una User Story si considera DONE solo quando soddisfa TUTTI i criteri obbligatori (✓) elencati di seguito, oltre ai propri Acceptance Criteria specifici. I criteri indicati da questo simbolo (○) si considerano opzionali.

Codice

1. **Compilazione pulita:** Il codice compila senza errori e i warning vengono gestiti in maniera appropriata ✓
2. **Naming conventions:** Le naming conventions Java sono rispettate ✓
3. **Version control:** Il codice è stato caricato (committato e pushato) sul branch principale del repository condiviso ✓
4. **Commenti significativi:** I commenti descrivono il "perché" delle scelte non ovvie, non ridondano il codice ✓

Design

5. **Alta coesione:** Alta coesione tra responsabilità delle classi ✓
6. **Basso accoppiamento:** Basso accoppiamento tra classi/package ✓
7. **Astrazione:** Uso di interfacce o classi astratte ove necessario ○
8. **Principio DRY:** Assenza di duplicazioni (Don't Repeat Yourself) ✓
9. **Rispetto dell'architettura:** Le modifiche rispettano l'architettura definita e non introducono violazioni dei layer/package ✓

Test

10. **Verifica dei criteri:** Ogni Acceptance Criterion della User Story è stato verificato con successo (tramite test automatizzati JUnit per la logica di backend e test manuali per l'interfaccia UI) ✓
11. **Copertura scenari:** Copertura dei Casi Limite essenziali: i test includono il percorso ideale e i principali scenari di errore previsti dalla card ✓
12. **Test superati:** Passaggio dei Test: tutti i test passano prima dell'integrazione nel branch principale ✓

Processo SCRUM

13. **Integrazione codice:** Codice integrato tramite merge/review nel repository GitHub ✓
14. **Code Review:** Revisione del codice effettuata da almeno un altro membro ✓
15. **Aggiornamento board:** Task spostato in Done su Trello ✓

16. **Artefatti Scrum:** Aggiornamento degli artefatti Scrum quando necessario ○

Interfaccia Utente

17. **Aggiornamento dinamico UI:** L'interfaccia aggiorna correttamente lo stato senza riavvio ✓
18. **Feedback visivo:** Messaggi di errore comprensibili e visibili ✓